TRANSITOS

SYSTEM DESIGN

Role Design & Operating Model

How ten roles, four business cycles and one set of records make TransitOS run the whole operation.

APPLIES TO

TransitOS v1.2.0
Expo app + TransitOS API

AUDIENCE

Head office, branch managers, fleet and finance staff

ISSUED

7 November 2026
Version 1.0

This document explains why the app is shaped the way it is: what each role is for, what it is accountable for, what it can and cannot see, and how the individual records combine into one view of the business. Every statement in it is checked against the shipping code.

Contents

Every entry below is a link — click it to jump straight to that section.

1 · The design principle 3
2 · The ten roles 3
3 · What each role is for 3
The platform tier 3
The company tier 4
The branch tier 4
The crew 4
The money tier 4
4 · The four cycles 4
Cycle 1 — Operations: getting the vehicle on the road 4
Cycle 2 — Money: what the trip actually earned 5
Cycle 3 — Fleet: keeping the asset serviceable 5
Cycle 4 — Settlement: paying the people 5
What ties them together 5
5 · How access actually works 6
6 · What the design deliberately refuses 6
7 · In one page 8

What this document is. TransitOS is one system that ten different people use for ten different reasons. A driver needs to know which bus is theirs. A conductor needs to know who paid. A branch manager needs to know whether the branch is profitable. An owner needs to know whether the company is.

This explains **how that is designed**: what each role is for, what it is accountable for, what it can and cannot see, and how the individual pieces — trips, manifests, cash-ups, settlements, alerts — combine into one view of the business.

Audience: the client team, from the owner down to the crew. **Companion documents:** the *User Manual* (how each screen works) and the *Onboarding Guide* (how to set up a branch and run a day).

1 · The design principle

Most apps are built for one person and then made to fit everyone else. TransitOS is built the other way round: **there is one record of the truth, and ten views onto it.**

A trip is one record. The manifest is one record. The cash-up is one record. The branch manager, the accountant and the owner are not looking at three different versions of the business — they are looking at the same records, through different doors.

Three ideas hold the whole thing together.

The record is the unit, not the screen. Every fact lives once. The trip's revenue is not stored on the trip *and* on the cash-up *and* in the settlement; it is derived from the manifest, and everything else reads it from there. That is why the numbers agree.

The role is the lens. Access is not decoration. What a person sees is decided by what they are accountable for, and the server enforces it on every request — not just by hiding a button in the app.

The scope is the boundary. A branch manager is powerful *within a branch* and powerless outside it. One branch's takings are invisible to another branch's team, at the database level, so it is a property of the system rather than a promise in a manual.

2 · The ten roles

Role	Who they are	What they are accountable for	What they see
Super Admin	The platform operator	The system itself: configuration, the permission matrix, audit	Everything, everywhere
Owner	The company's principal	The whole business; sets policy	Everything, everywhere
General Manager	Runs the company day to day	Every branch, every fleet, every trip	Company-wide
Operations Manager	Plans and dispatches	Trips, schedules, incidents, transfers	Company-wide operations
Fleet Manager	Keeps the vehicles roadworthy	Vehicles, fuel, servicing, documents, licences	Company-wide fleet
Branch Manager	Runs one depot	Their branch's people, trips, fleet and cash	Their branch only
Chief Accountant	The company's finance	Cash-ups, expenses, settlements, reports	Company-wide money
Branch Accountant	The branch's finance	Their branch's money and its settlements	Their branch only
Driver	Drives the vehicle	The pre-trip check; the vehicle and its paperwork	Their own trips only
Conductor	Sells and collects	The manifest and the cash-up	Their own trips only

Two facts make this table work. **Seniority is a hierarchy** — nobody can grant a role more senior than their own, so a branch manager cannot mint a general manager. **Scope is separate from seniority** — being senior buys you breadth, not access to another branch's money.

3 · What each role is for

The platform tier

Super Admin exists because a system with ten roles needs somebody who can change the rules. They own the permission matrix — the table that says what each role may do to each kind of record — and they are the only role that can read it and the audit log. Their dashboard is the company view: revenue, trips, fleet, variance, and the counters that say what needs attention today.

Owner is the business person at the top: the same view of the same data, with the authority to create branches and users and to approve anything the general manager could not. The owner and the super admin see the same dashboard — deliberately. The difference is what they are *allowed to change*, not what they are allowed to *see*.

The company tier

General Manager is the hinge between the platform and the branches. Everything an operations manager and a fleet manager can do, the GM can do too — across every branch. The GM is also the tier that creates branches and appoints branch managers, so the org chart is built from here.

Operations Manager is the day. They build the schedule: which vehicle, which route, which driver, which conductor, and when. They own incidents and vehicle transfers, and they watch the live board. Their dashboard is the operational picture — trips today, the active fleet, who is on duty, what is open. Deliberately no revenue: the money is finance's view, and mixing the two is how a fleet manager ends up arguing about a variance they cannot act on.

Fleet Manager owns the asset. Vehicles in and out of service, fuel logged against a distance, servicing scheduled and recorded, documents and licences kept in date. Their dashboard is the fleet: how many vehicles are available or on the road, how many are in maintenance, whose paperwork expires in the next thirty days.

The branch tier

Branch Manager runs a depot. They set up their branch's vehicles, crew and routes, create the accounts for their own staff, dispatch the day, and are accountable for their branch's money. Their dashboard is their branch — the branch manager sees *2 of 3* vehicles where the general manager sees *3 of 5*, because the server scopes the figures to their branch. They are also the only tier below the GM that can create user accounts — the permission matrix itself is the top tier's alone.

Branch Accountant owns the branch's money: expenses, cash-ups and settlements for their own branch. They see a financial position and a queue of cash-ups waiting on them, and nothing about the fleet.

The crew

Driver and **Conductor** are the two people the business actually happens through. They are deliberately the narrowest roles in the system, and the app is built around that.

A driver sees **their trips and their state**. No revenue, no fleet, no company figures — because a driver cannot act on any of them, and showing them a number they cannot influence is how a system stops being trusted. The driver's job in the app is short: the pre-trip checklist, the trip, the incident report, and the QR scan.

A conductor sees **their trips, their manifest and their cash-up**. The manifest is the record of who travelled and what they paid; the cash-up is the declaration of what was collected. Both are the conductor's own work, and the app makes both of them easy and hard to get wrong.

The money tier

Chief Accountant approves money across the company. They see the company's financial position, the cash-ups waiting for approval, and the reports. They cannot dispatch a vehicle or schedule a trip — they can approve the money that comes out of one.

Branch Accountant is the same job for one branch. They see their branch's expenses and cash-ups and their branch's settlements, and nothing belonging to another branch.

4 · The four cycles

Every record in TransitOS belongs to one of four cycles. Each cycle has an owner, a sequence, and a closing event that hands the result to the next cycle.

Cycle 1 — Operations: getting the vehicle on the road

```
Fuel logged → Pre-trip checklist → Trip created → Boarding
            → Departed → Arrived → Closed → Cashed up
```

The **fleet manager** logs fuel and schedules servicing. The **driver** walks the ten-point pre-trip checklist against the vehicle. The **operations manager** creates the trip — route, vehicle, driver, conductor, departure, arrival. The **branch manager** (or the operations manager) moves the trip through its stages: *Start boarding*, *Confirm departure*, *Confirm arrival*, *Close trip*. The server refuses an out-of-order move, and records who moved it and when.

The trip's arrival time and the actual departure are stamped as the stages are confirmed, so the record shows what happened rather than what was planned.

Cycle 2 — Money: what the trip actually earned

```
Manifest (passengers and cargo) ➝ Expected revenue
   ➝ Conductor counts the cash ➝ Cash-up saved (draft)
   ➝ Submitted ➝ Approved or rejected ➝ Trip marked cashed up
```

This is the cycle the app was rebuilt around, and it is the one where the role design matters most.

The **manifest** is the source of every revenue figure in the system. The trip's revenue is the sum of its manifest fares. The cash-up's *expected total* is the same sum, recomputed by the server at the moment of submission. The *variance* — declared minus expected — is the difference between what the conductor counted and what the manifest said should have been collected.

The **conductor** owns the manifest: + *Passenger* and + *Cargo* while the trip is still planned or boarding, and the bin icon to remove someone who did not travel. It locks when the vehicle departs, because a passenger cannot board a bus that has already left.

The **conductor** then declares the takings — counts of ₦1,000, ₦500, ₦200, ₦100, ₦50 notes and coins. *Save cash-up* records a **draft**, visible only to them, which they can correct and save again. *Submit for approval* hands it to finance.

The **chief accountant** (or **branch accountant**) approves or rejects. Approval computes the final variance, marks the trip *cashed up*, and links the cash-up to the trip. A rejection carries a reason, and the cash-up returns to the conductor to be corrected and submitted again.

Cycle 3 — Fleet: keeping the asset serviceable

```
Vehicle created ➝ Fuel logs ➝ Maintenance records ➝ Documents
                ➝ Transfers between branches ➝ Alerts when due
```

The **fleet manager** owns this cycle end to end. Fuel logs feed the efficiency calculation; maintenance records carry parts, labour cost and the next due distance; vehicle documents (registration, insurance, inspection) and driver licences are tracked with expiry dates. Vehicles move between branches by a **transfer** — initiated, confirmed, and returned — so the asset never teleports between branches without a record.

The **alerts engine** runs on a timer and raises what is due: vehicle documents expiring, driver licences expiring, maintenance due or overdue, and low fuel. It is the part of the system that acts without being asked.

Cycle 4 — Settlement: paying the people

```
Closed and cashed-up trips ➝ Settlement lines (revenue, expenses, crew pay)
                           ➝ Draft ➝ Submitted ➝ Approved ➝ Paid
```

The **chief accountant** generates a settlement for a period — a branch and a month, say. One line per completed trip, with the gross revenue, the trip's expenses, and what the driver and conductor are owed at the branch's pay rates. The settlement is a draft, then submitted, approved and paid.

This is where the four cycles meet. A settlement is only as good as the trips behind it, and the trips are only as good as the manifest and the cash-up. The chain is why a cash-up cannot be faked: change the manifest and the expected figure changes; change the cash-up and the variance changes; the settlement follows both.

What ties them together

```
Branch ──┬── Vehicle ── Fuel · Maintenance · Documents
         ├── Driver / Conductor ── Licences · Badges
         ├── Route ── Distance · Fares · Duration
         └── Trip ──┬── Manifest ──► Expected revenue
                    ├── Expenses ────► Trip cost
                    ├── Cash-up ─────► Declared · Variance
                    └── Position ────► Live board
                                  │
                                  └──► Settlement ──► Crew pay
```

Every arrow is a real foreign key in the database, not a convention. That is why a branch's settlement can be traced back to the individual passenger who paid ₦2,500 on a particular trip, and why the figures agree at every level.

5 · How access actually works

Three mechanisms, in the order they apply.

Role gates — every endpoint declares which roles may call it. The crew directory, for instance, is closed to drivers and conductors: a crew member has no business browsing the staff list. A conductor can read and edit the cash-ups **they** declared — their own, not the company's — and a request for someone else's returns *404*, not *403*, so it cannot be discovered by guessing an identifier.

The permission matrix — nineteen kinds of record (branches, users, vehicles, drivers, conductors, routes, trips, fuel, maintenance, expenses, cash-ups, reports, audit log, notifications, transfers, incidents, inspections, vehicle documents, role permissions), each with create, read, update, delete, approve and export. Every role has a row; the Super Admin, Owner and General Manager hold everything, and the rest are carved deliberately:

Resource	Who does what
Branches	Created by the top tier only; read by the branch tier; invisible to crew
Users	Created by the top tier and the branch manager; read by the branch accountant
Vehicles, drivers, conductors	Created by the branch, operations and fleet managers; read-only for finance and crew
Routes, transfers, vehicle documents	Created by the branch, operations and fleet managers; the top tier may delete
Trips	Scheduled by the branch, operations and fleet managers; read by everyone
Fuel	Logged by the fleet and branch managers — **and by the driver at the pump**
Maintenance, inspections	Owned by the fleet and branch managers; the driver records his own checklist
Expenses	Recorded by the branch accountant and the top tier; approved by the chief accountant and above
Cash-ups	Declared by the conductor and the branch accountant; **approved** by the chief accountant, the branch manager and the top tier
Reports	Read by every management and finance role; not by crew
Audit log, role permissions	The top tier only
Settlements	Read by the branch and chief accountants; created by the chief accountant

Scope — every role has one of three scopes, and it is recorded with the role rather than guessed from it:

Scope	Roles	Means
`all`	Super Admin, Owner, General Manager, Operations Manager, Fleet Manager, Chief Accountant	every branch
`branch`	Branch Manager, Branch Accountant, Fleet Manager	their own branch only
`own`	Driver, Conductor	their own trips and their own declarations

For a scoped role the server adds the restriction to every query and every write. A branch accountant in Lagos cannot open a cash-up belonging to Abuja, and cannot read one by guessing its id. This is enforced in the database query, not in the interface.

The screen access follows the same logic: twenty-one screens, each with its own list of roles. The Super Admin, the Owner and the General Manager open all twenty-one; the Branch Manager nineteen; the Fleet Manager fourteen; the Operations Manager thirteen; the Chief Accountant four; the Branch Accountant six; **the Driver and the Conductor four each**. Where a screen would show a role information it cannot act on, the role does not get the screen — and where a card on a dashboard is not the role's business, the card is not on the dashboard.

6 · What the design deliberately refuses

A crew member does not see company figures. Not blurred, not greyed out — absent. A driver whose dashboard shows the company's revenue learns to ignore the dashboard.

The crew directory is closed. Drivers and conductors cannot browse the staff list. Names reach the screens that need them (a trip's crew) by travelling with the record, not by a lookup the crew cannot make.

Nobody grants a role above their own. The hierarchy is enforced on assignment, so access cannot be escalated by an invitation.

A trip cannot skip a stage, and a closed trip cannot be reopened. The state machine is on the server, and every move is audited.

Cash is declared by one person and approved by another. The conductor who counts the cash never approves it.

Every mutation is written to the audit log — who, what, when, and the record before and after — and only the top tier can read it.

7 · In one page

Read straight out of the permission matrix. ● means the role holds that permission; — means it does not.

	Super Admin	Owner	General Manager	Operations Mgr	Fleet Manager	Branch Manager	Chief Accountant	Branch Accountant	Driver	Conductor
	●	●	●	—	—	—	—	—	—	—
Creates a user account	●	●	●	—	—	●	—	—	—	—
	●	●	●	●	●	●	—	—	—	—
Schedules trips	●	●	●	●	●	●	—	—	—	—
	●	●	●	●	●	●	—	—	—	—
Records the manifest	●	●	●	●	●	●	—	—	—	—
	●	●	●	—	●	●	—	—	●	—
Records maintenance	●	●	●	—	●	●	—	—	—	—
	●	●	●	●	●	●	—	—	●	—
Records an incident	●	●	●	●	●	●	—	—	●	●
	●	●	●	—	—	●	●	●	—	—
Declares a cash-up	●	●	●	—	—	—	—	●	—	●
	●	●	●	—	—	●	●	—	—	—
Reads the reports	●	●	●	●	●	●	●	●	—	—
	●	●	●	●	—	●	—	—	—	—

Two rows carry the design. **"Declares a cash-up" and "Approves a cash-up" never share a role** — except at the top, where the business owner is accountable for everything. The person who counts the cash never approves it. And **"Records the pre-trip check" belongs to the driver**, because the person who walks around the vehicle is the only one who can honestly say it is safe.

The shape of that table is the whole design: **the people who do the work record it, the people who are accountable for it approve it, and the people who own the business see it all.** Every role in TransitOS is one of those three things — and the app is built so that each of them can do exactly their part and no more.

A note on two rows. "Records the manifest" is shown against the roles that can write a trip's passenger list — which is the operations and branch tiers plus the conductor, whose own trip they record. "Records expenses" and "Declares a

cash-up" are separate: the branch accountant records an expense, the conductor declares a cash-up, and the two never overlap in the crew.